

ASSIGNMENT NO.7: Subqueries and Nested Queries

Objective:

The objective of this practical is to help students understand how to use subqueries and nested queries in MySQL. Students will learn how to write subqueries in SELECT and WHERE clauses, use correlated subqueries, solve problems using nested queries, and apply subqueries in real-world query scenarios.

Introduction to Subqueries and Nested Queries

A subquery is a query written inside another SQL query. It is also called a nested query. The inner query is executed first, and the result of the inner query is used by the outer query. Subqueries are useful when one query depends on the result of another query.

Basic Terms

Term	Meaning
Outer Query	The main query that uses the result of another query
Inner Query	The query written inside another query
Subquery	A query inside another SQL query
Nested Query	Another name for subquery
Correlated Subquery	A subquery that depends on the outer query

Simple Example Requirement

Find students who scored more than the average marks.

Step	Action
1	Find the average marks of all students
2	Compare each student's marks with the average
3	Display students whose marks are greater than the average

Example Query

```
SELECT StudentName, Marks FROM StudentResult WHERE Marks > (SELECT  
AVG(Marks) FROM StudentResult);
```

Explanation

Query Part	Purpose
SELECT AVG(Marks) FROM StudentResult	Inner query calculates average marks
WHERE Marks > (...)	Outer query filters students above average
SELECT StudentName, Marks	Displays required student details

Creating Sample Tables and Inserting Records

For this practical, we will use two tables: Course and StudentResult.

Course Table Structure

Column Name	Data Type	Constraint
CourseID	INT	PRIMARY KEY
CourseName	VARCHAR(50)	NOT NULL
Fees	DECIMAL(10,2)	NOT NULL

StudentResult Table Structure

Column Name	Data Type	Constraint
StudentID	INT	PRIMARY KEY
StudentName	VARCHAR(50)	NOT NULL
CourseID	INT	FOREIGN KEY
City	VARCHAR(30)	
Marks	INT	

MySQL Code to Create Database and Tables

```
CREATE DATABASE CollegeDB;
USE CollegeDB;
CREATE TABLE Course ( CourseID INT PRIMARY KEY, CourseName VARCHAR(50) NOT
NULL, Fees DECIMAL(10,2) NOT NULL );
CREATE TABLE StudentResult ( StudentID INT PRIMARY KEY, StudentName
VARCHAR(50) NOT NULL, CourseID INT, City VARCHAR(30), Marks INT, FOREIGN
KEY (CourseID) REFERENCES Course(CourseID) );
```

MySQL Code to Insert Records

```
INSERT INTO Course
(CourseID, CourseName, Fees)
VALUES
(101, 'BSc IT', 45000.00),
(102, 'BCA', 40000.00),
(103, 'BSc CS', 42000.00),
(104, 'Data Science', 55000.00);

INSERT INTO StudentResult
(StudentID, StudentName, CourseID, City, Marks)
VALUES
(1, 'Rahul Sharma', 101, 'Mumbai', 78),
(2, 'Priya Patil', 102, 'Pune', 85),
(3, 'Aman Verma', 103, 'Nashik', 67),
(4, 'Sneha Joshi', 101, 'Mumbai', 91),
(5, 'Karan Mehta', 102, 'Thane', 74),
(6, 'Neha Shah', 101, 'Pune', 88),
(7, 'Rohan Desai', 103, 'Mumbai', 69),
(8, 'Meera Kulkarni', 102, 'Nashik', 82),
(9, 'Arjun Nair', 101, 'Thane', 59),
(10, 'Divya More', 102, 'Mumbai', 93);
```

Course Table

CourseID	CourseName	Fees
101	BSc IT	45000.00
102	BCA	40000.00
103	BSc CS	42000.00
104	Data Science	55000.00

StudentResult Table

StudentID	StudentName	CourseID	City	Marks
1	Rahul Sharma	101	Mumbai	78
2	Priya Patil	102	Pune	85
3	Aman Verma	103	Nashik	67
4	Sneha Joshi	101	Mumbai	91
5	Karan Mehta	102	Thane	74
6	Neha Shah	101	Pune	88
StudentID	StudentName	CourseID	City	Marks
7	Rohan Desai	103	Mumbai	69
8	Meera Kulkarni	102	Nashik	82
9	Arjun Nair	101	Thane	59
10	Divya More	102	Mumbai	93

Writing Subqueries in WHERE Clause

A subquery in the WHERE clause is used when the condition depends on the result of another query.

Syntax

```
SELECT column_names FROM table_name WHERE column_name operator (SELECT column_name FROM table_name WHERE condition);
```

Example 1: Display Students Who Scored More Than Average Marks

```
SELECT StudentName, Marks FROM StudentResult WHERE Marks > (SELECT AVG(Marks) FROM StudentResult );
```

Output

StudentName	Marks
Priya Patil	85
Sneha Joshi	91
Neha Shah	88
Meera Kulkarni	82
Divya More	93

Explanation

Step	Description
1	Inner query calculates average marks of all students

2	Outer query compares each student's marks with the average
3	Students scoring above average are displayed

Example 2: Display Students Enrolled in BCA Course

SELECT StudentID, StudentName, CourseID, Marks FROM StudentResult WHERE CourseID = (SELECT CourseID FROM Course WHERE CourseName = 'BCA'); **Output**

StudentID	StudentName	CourseID	Marks
2	Priya Patil	102	85
5	Karan Mehta	102	74
8	Meera Kulkarni	102	82
10	Divya More	102	93

Where Clause Subquery Summary

Use Case	Query Logic
Students above average marks	Marks > (SELECT AVG(Marks) ...)
Students from a specific course	CourseID = (SELECT CourseID...)
Students in high-fee courses	CourseID IN (SELECT CourseID...)

4. Writing Subqueries in SELECT Clause

A subquery in the SELECT clause is used to display a calculated or related value along with each row.

This is useful when we want to display normal row data with an additional calculated value.

Example 1: Display Each Student with Overall Average Marks

SELECT StudentName, Marks, (SELECT AVG(Marks) FROM StudentResult) AS AverageMarks FROM StudentResult;

Output

StudentName	Marks	AverageMarks
Rahul Sharma	78	78.6000
Priya Patil	85	78.6000
Aman Verma	67	78.6000
Sneha Joshi	91	78.6000
Karan Mehta	74	78.6000
Neha Shah	88	78.6000
Rohan Desai	69	78.6000
Meera Kulkarni	82	78.6000
Arjun Nair	59	78.6000
Divya More	93	78.6000

Example 2: Display Student Name with Course Name using Subquery

SELECT StudentName, CourseID, (SELECT CourseName FROM Course WHERE Course.CourseID = StudentResult.CourseID) AS CourseName FROM StudentResult;

Output

StudentName	CourseID	CourseName
Rahul Sharma	101	BSc IT
Priya Patil	102	BCA
Aman Verma	103	BSc CS
Sneha Joshi	101	BSc IT
Karan Mehta	102	BCA
Neha Shah	101	BSc IT
Rohan Desai	103	BSc CS
Meera Kulkarni	102	BCA
Arjun Nair	101	BSc IT
Divya More	102	BCA

Explanation

Query Part	Purpose
Outer query	Displays student records
Inner query	Finds related course name
AS CourseName	Gives a readable column name
Subquery in SELECT	Adds extra calculated or related information

Correlated Subqueries and Real-World Query Scenarios

A **correlated subquery** is a subquery that depends on the outer query. It uses values from the outer query and executes for each row processed by the outer query.

Normal Subquery vs Correlated Subquery

Type	Meaning	Execution
Normal Subquery	Inner query can run independently	Executes once
Correlated Subquery	Inner query depends on outer query	Executes for each row
Nested Query	General name for query inside another query	Depends on query type

Example 1: Display Students Who Scored More Than Their Course Average

```
SELECT StudentID, StudentName, CourseID, Marks FROM StudentResult S1 WHERE  
Marks > (SELECT AVG(Marks)FROM StudentResult S2 WHERE S2.CourseID =  
S1.CourseID);
```

Output

StudentID	StudentName	CourseID	Marks
4	Sneha Joshi	101	91
6	Neha Shah	101	88
10	Divya More	102	93
7	Rohan Desai	103	69

Explanation

Query Part	Description
S1	Alias for outer StudentResult table
S2	Alias for inner StudentResult table
S2.CourseID = S1.CourseID	Compares records from the same course
Inner query	Calculates average marks course-wise
Outer query	Displays students above their course average

Example 2: Display Courses Having Students

Output

CourseID	CourseName
101	BSc IT
102	BCA
103	BSc CS

Example 3: Display Courses Having No Students

```
SELECT CourseID, CourseName FROM Course C WHERE NOT EXISTS (SELECT 1 FROM StudentResult S WHERE S.CourseID = C.CourseID);
```

Output

CourseID	CourseName
104	Data Science

Real-World Query Scenarios

Scenario	Requirement	Suitable Query Type
Academic performance	Find students above average marks	Subquery in WHERE
Course report	Display students from selected course	Subquery in WHERE
Comparison report	Show each student with overall average	Subquery in SELECT
Course-wise analysis	Find students above their course average	Correlated subquery
Admission tracking	Find courses having enrolled students	EXISTS
Empty course report	Find courses with no students	NOT EXISTS

Final Summary Table

Concept	Purpose	Example Use
Subquery	Query inside another query	Find students above average marks
Concept	Purpose	Example Use
Nested query	Another name for subquery	Query inside WHERE
Subquery in WHERE	Filters rows using result of inner query	Marks greater than average

Subquery in SELECT	Displays calculated or related value	Average marks with each student
Correlated subquery	Inner query depends on outer query	Students above course average
EXISTS	Checks whether related rows exist	Courses having students
NOT EXISTS	Checks whether related rows do not exist	Courses without students

Questions:

Q1) Create a MySQL database named CollegeDB and create two tables: Course and StudentResult.

```
CREATE DATABASE CollegeDB;
```

```
USE CollegeDB;
```

```
CREATE TABLE Course (
```

```
    CourseID INT PRIMARY KEY,
```

```
    CourseName VARCHAR(50) NOT NULL,
```

```
    Fees DECIMAL(10,2) NOT NULL
```

```
);
```

```
CREATE TABLE StudentResult (
```

```
    StudentID INT PRIMARY KEY,
```

```
    StudentName VARCHAR(50) NOT NULL,
```

```
    CourseID INT,
```

```
    City VARCHAR(30),
```

```
    Marks INT,
```

```
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
```

```
);
```

```
INSERT INTO Course VALUES
```

```
(101,'BSc IT',45000.00),
```

```
(102,'BCA',40000.00),
```

```
(103,'BSc CS',42000.00),
```

```
(104,'Data Science',55000.00);
```

INSERT INTO StudentResult VALUES

(1,'Rahul Sharma',101,'Mumbai',78),

(2,'Priya Patil',102,'Pune',85),

(3,'Aman Verma',103,'Nashik',67),

(4,'Sneha Joshi',101,'Mumbai',91),

(5,'Karan Mehta',102,'Thane',74),

(6,'Neha Shah',101,'Pune',88),

(7,'Rohan Desai',103,'Mumbai',69),

(8,'Meera Kulkarni',102,'Nashik',82),

(9,'Arjun Nair',101,'Thane',59),

(10,'Divya More',102,'Mumbai',93);

The screenshot shows the phpMyAdmin web interface. The browser address bar displays 'localhost/phpmyadmin/index.php?route=/sql&pos=0&db=collagedb&table=course'. The interface includes a sidebar with a database tree on the left and a main content area on the right. The main area shows the 'course' table with 5 rows. The table has columns: courseid, coursename, duration, fees, DepartmentID, Tier, and RankGroup. The data rows are as follows:

courseid	coursename	duration	fees	DepartmentID	Tier	RankGroup
101	SQL	5	50000.00	1	VIP Program	Most Affordable
102	B.SCIt	2	82000.00	2	VIP Program	Premium Tier
103	BBA	2	30000.00	1	Standard Program	Most Affordable
104	BCA	3	40000.00	2	Standard Program	Premium Tier
105	DATA SCIENCE	1	60000.00	1	VIP Program	Most Affordable

Course Table

Column Name	Data Type	Constraint
CourseID	INT	PRIMARY KEY
CourseName	VARCHAR(50)	NOT NULL
Fees	DECIMAL(10,2)	NOT NULL

StudentResult Table

Column Name	Data Type	Constraint
StudentID	INT	PRIMARY KEY

StudentName	VARCHAR(50)	NOT NULL
CourseID	INT	FOREIGN KEY
City	VARCHAR(30)	
Marks	INT	

The screenshot shows the phpMyAdmin interface for a database named 'collegedb'. The 'studentresult' table is selected, and its structure is displayed. The table has the following columns: StudentID, StudentName, Course, City, Marks, and FeesPaid. The table contains 10 records, which are listed in the main content area. The interface also includes a sidebar with a database tree, a top navigation bar, and a main content area with a SQL query editor and a table view.

StudentID	StudentName	Course	City	Marks	FeesPaid
1	Amit Sharma	BCA	Pune	85	45000.00
2	Priya Patel	BCA	Mumbai	92	48000.00
3	Rahul Verma	BSc IT	Pune	76	42000.00
4	Neha Singh	BCA	Nashik	88	45000.00
5	Rohan Mehta	BSc IT	Mumbai	69	40000.00
6	Sneha Joshi	BCA	Nagpur	95	50000.00
7	Karan Shah	BSc CS	Pune	81	43000.00
8	Pooja Desai	BSc CS	Surat	73	41000.00
9	Vikas Kumar	BCA	Delhi	64	39000.00
10	Anjali Rao	BSc IT	Bangalore	90	47000.00

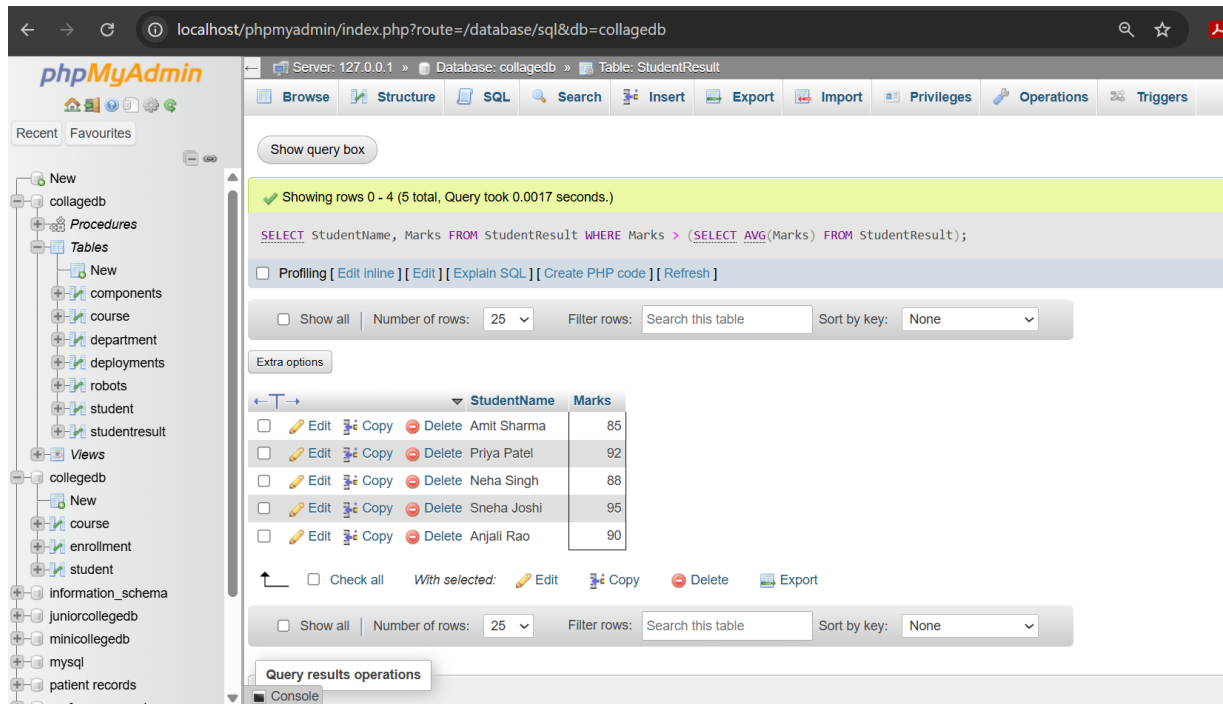
Insert at least 4 records into Course and 10 records into StudentResult.

Q2) Write a MySQL query to display students whose marks are greater than the average marks of all students.

SELECT StudentName, Marks

FROM StudentResult

WHERE Marks > (SELECT AVG(Marks) FROM StudentResult);



The output should contain:

Column
StudentName
Marks

Q3) Write a MySQL query to display all students enrolled in the BCA course using a subquery.

SELECT StudentID, StudentName, CourseID, Marks

FROM StudentResult

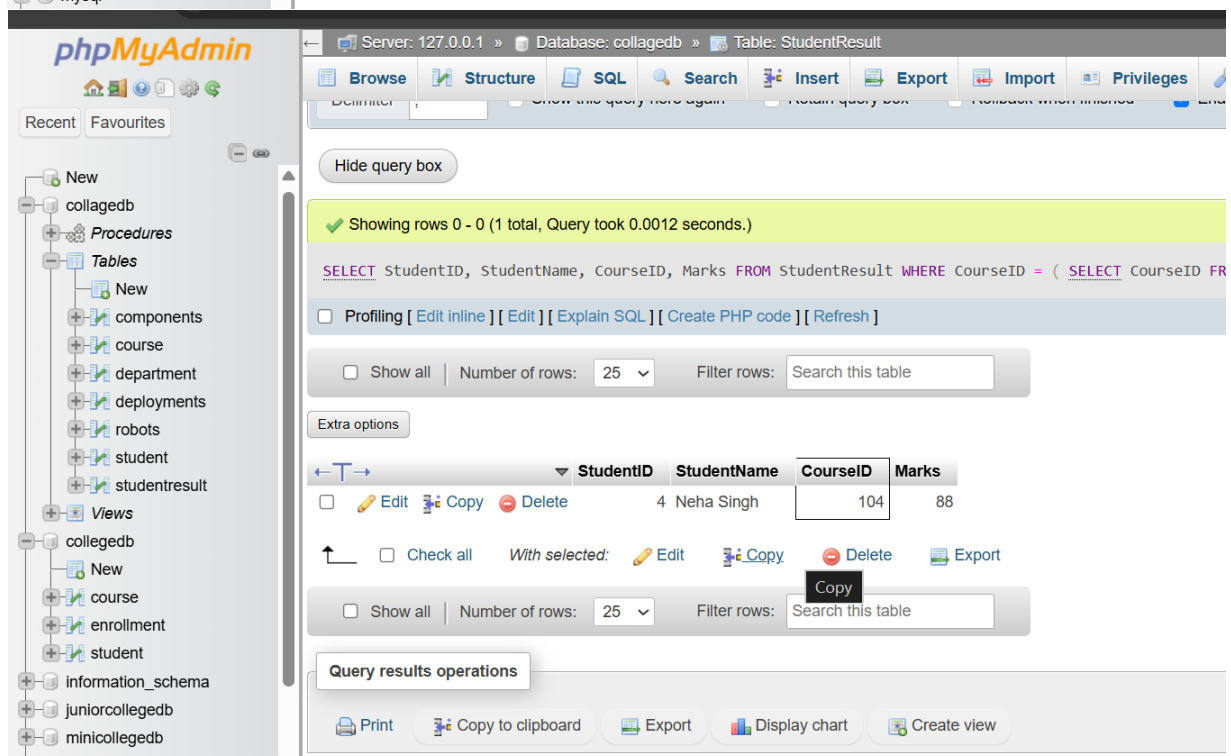
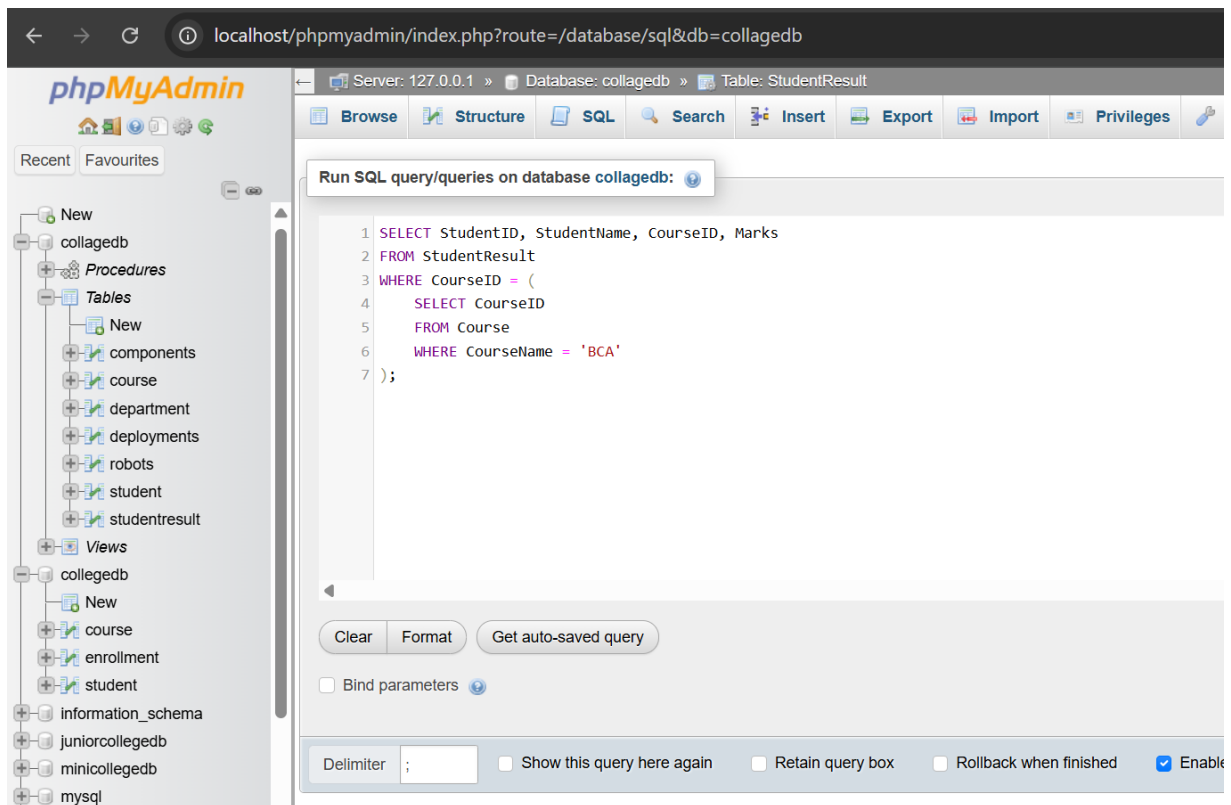
WHERE CourseID = (

SELECT CourseID

FROM Course

WHERE CourseName = 'BCA'

);



The output should contain:

Column
StudentID
StudentName

Column
CourseID
Marks

Q4) Write a MySQL query to display each student with the overall average marks using a subquery in the SELECT clause.

SELECT StudentName, Marks,

(SELECT AVG(Marks) FROM StudentResult) AS AverageMarks

FROM StudentResult;

The screenshot shows the phpMyAdmin web interface. The browser address bar displays 'localhost/phpmyadmin/index.php?route=/database/sql&db=collegedb'. The interface includes a sidebar with a database tree structure showing 'collegedb' and its tables: 'components', 'course', 'department', 'deployments', 'robots', 'student', and 'studentresult'. The main panel is titled 'Run SQL query/queries on database collegedb:' and contains the following SQL query:

```
1 SELECT StudentName, Marks,
2     (SELECT AVG(Marks) FROM StudentResult) AS AverageMarks
3 FROM StudentResult;
```

Below the query editor, there are buttons for 'Clear', 'Format', and 'Get auto-saved query'. There is also a checkbox for 'Bind parameters' which is currently unchecked. At the bottom, there is a 'Delimiter' dropdown set to ';' and three checkboxes: 'Show this query here again' (unchecked), 'Retain query box' (unchecked), and 'Rollback when finished' (checked).

localhost/phpmyadmin/index.php?route=/database/sql&db=collagedb

Server: 127.0.0.1 » Database: collagedb » Table: StudentResult

Browser Structure SQL Search Insert Export Import Privileges

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

StudentName	Marks	AverageMarks
Amit Sharma	85	81.3000
Priya Patel	92	81.3000
Rahul Verma	76	81.3000
Neha Singh	88	81.3000
Rohan Mehta	69	81.3000
Sneha Joshi	95	81.3000
Karan Shah	81	81.3000
Pooja Desai	73	81.3000
Vikas Kumar	64	81.3000
Anjali Rao	90	81.3000

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Query results operations

Print Copy to clipboard Export Display chart Create view

Console

The output should contain:

Column
StudentName
Marks
AverageMarks

Q5) Write a MySQL query using a correlated subquery to display students who scored more than their own course average.

SELECT S1.StudentID, S1.StudentName, S1.CourseID, S1.Marks

FROM StudentResult S1

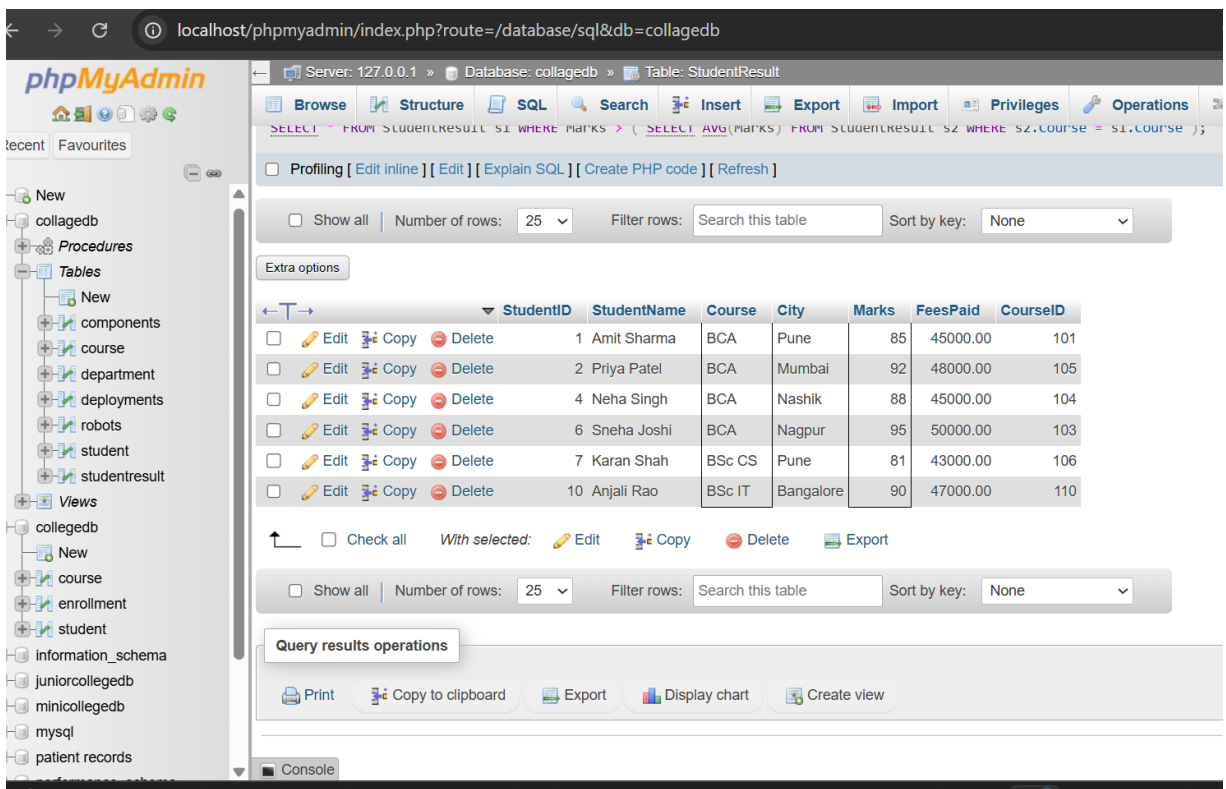
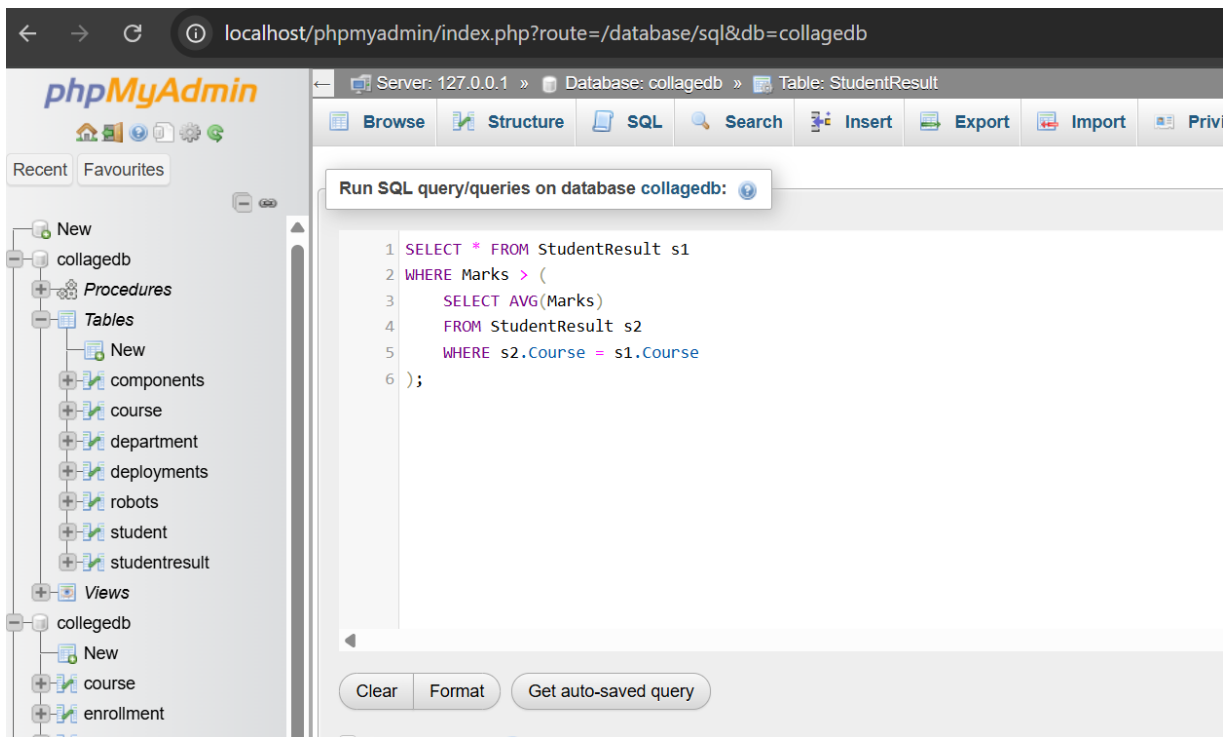
WHERE S1.Marks > (

SELECT AVG(S2.Marks)

FROM StudentResult S2

WHERE S2.CourseID = S1.CourseID

);



The output should contain:

Column
StudentID
StudentName

CourseID
Marks

Q6) Write MySQL queries to display courses having students and courses having no students.

-- Courses having at least one student

SELECT C.CourseID, C.CourseName

FROM Course C

WHERE EXISTS (

 SELECT 1

 FROM StudentResult S

 WHERE S.CourseID = C.CourseID

);

-- Courses having no students

SELECT C.CourseID, C.CourseName

FROM Course C

WHERE NOT EXISTS (

 SELECT 1

 FROM StudentResult S

 WHERE S.CourseID = C.CourseID

);

localhost/phpmyadmin/index.php?route=/database/sql&db=collagedb

phpMyAdmin

Server: 127.0.0.1 » Database: collagedb » Table: Course

Browse Structure SQL Search Insert Export Import Pr

Show query box

Showing rows 0 - 4 (5 total, Query took 0.0013 seconds.)

-- Courses having at least one student `SELECT C.CourseID, C.CourseName FROM Course C WHERE E C.CourseID );`

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: No

Extra options

	CourseID	CourseName
☐ Edit Copy Delete	101	SQL
☐ Edit Copy Delete	102	B.SCIt
☐ Edit Copy Delete	103	BBA
☐ Edit Copy Delete	104	BCA
☐ Edit Copy Delete	105	DATA SCIENCE

☐ Check all With selected: Edit Copy Delete Export

Use:

Operator

EXISTS

NOT EXISTS

Assignment Evaluation

- 0: Not Done []
- 1: Incomplete []
- 2: Late Complete []
- 3: Needs Improvement []
- 4: Complete []
- 5: Well Done []

Signature of Instructor: _____

